



Sistema de Assinatura Digital: RSA-OAEP e RSA-PSS

Trabalho de Implementação 2 – CIC0201

Ana Luísa Reis Nascente Gabriel de Sousa
Marina Pimentel Moreno Pedro de Paula Campos

29 de setembro de 2026

Cifragem (Parte II)

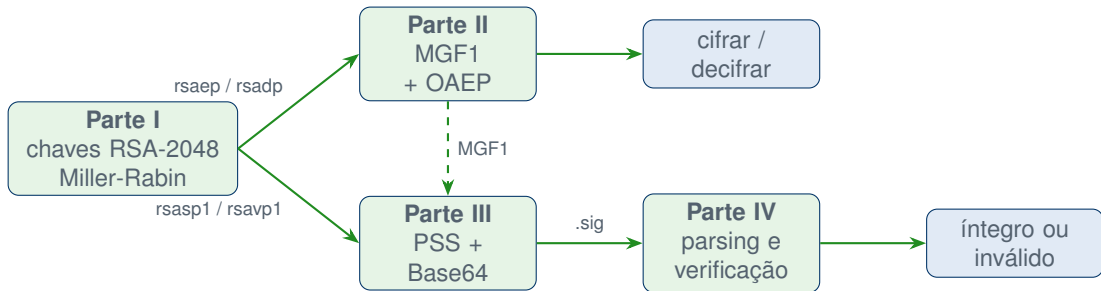
- ▶ Objetivo: **sigilo**.
- ▶ Quem envia cifra com a chave **pública**.
- ▶ Só o dono da chave **privada** decifra.
- ▶ Esquema: **RSA-OAEP** (até 190 bytes).

Assinatura (Parte III)

- ▶ Objetivo: **autenticidade e integridade**.
- ▶ O dono assina com a chave **privada**.
- ▶ Qualquer um verifica com a chave **pública**.
- ▶ Esquema: **RSA-PSS** sobre o SHA3-256.

Restrição do roteiro

Chaves, Miller-Rabin, RSA, OAEP, MGF1 e PSS feitos pelo grupo; bibliotecas prontas só para SHA3-256 e para conferir a saída.



Python 3, só biblioteca padrão: `secrets` (aleatoriedade), `hashlib.sha3_256`, `base64`, `json`.

CLI integrada: gerar-chaves, cifrar, decifrar, assinar, verificar.

Par de chaves (FIPS 186-5, A.1.1)

1. p, q de 1024 bits, distintos, $|p - q| > 2^{412}$
2. $n = pq$ $\lambda(n) = \text{mmc}(p - 1, q - 1)$
3. $e = 65537$ $d = e^{-1} \bmod \lambda(n)$
4. CRT: d_p, d_q, q_{inv} (decifra $\approx 4\times$ mais rápido)

Aritmética própria: Euclides estendido, inverso modular e exponenciação por quadrados sucessivos.

Miller-Rabin

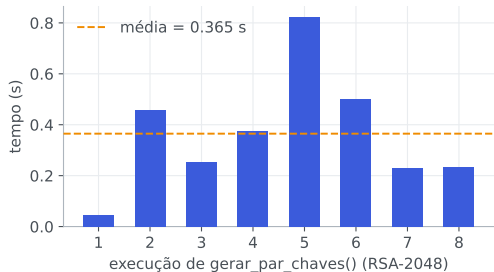
$n - 1 = 2^s d$, base a aleatória:

$x = a^d \bmod n$; se $x \in \{1, n - 1\}$ passa, senão eleva ao quadrado até $s - 1$ vezes.

- ▶ Crivo com primos < 1000 antes do teste.
- ▶ **4 rodadas** para primos de 1024 bits: Tabela B.1 da FIPS 186-5, erro $\leq 2^{-100}$.

Parte I – Resultados

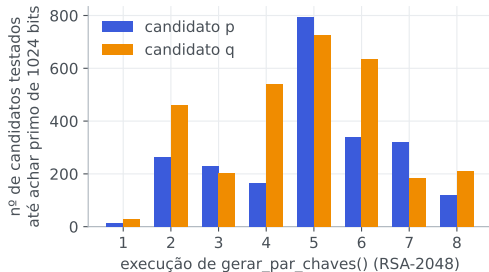
Geração de chaves RSA-2048



Tempo por geração (8 execuções)

0,365 s

tempo médio



Candidatos testados até achar cada primo

11/11

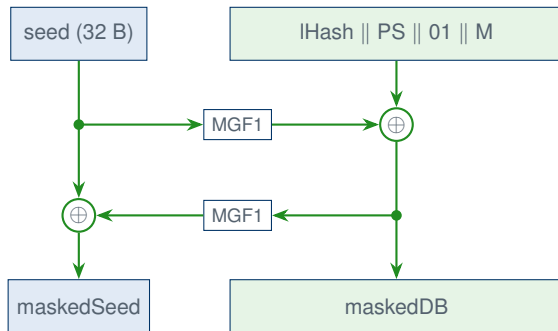
testes da Parte I

14–794

candidatos por primo

Parte II – RSA-OAEP

Codificação: Feistel de duas rodadas com MGF1



$EM = 00 || \text{maskedSeed} || \text{maskedDB} \text{ (256 bytes)} \rightarrow c = EM^e \bmod n$

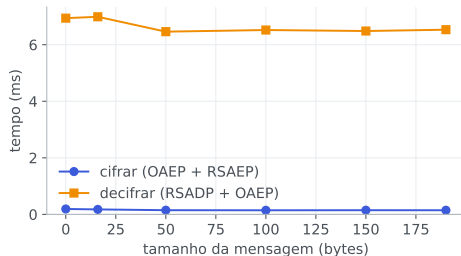
- ▶ **MGF1:**
 $\text{SHA3}(Z || 0) || \text{SHA3}(Z || 1) || \dots$
- ▶ **seed** aleatória do `secrets`:
cifragem probabilística.
- ▶ As duas máscaras amarram tudo: mudar 1 bit embaralha o bloco inteiro.
- ▶ Limite: $k - 2 \cdot hLen - 2 = \mathbf{190 \text{ bytes}}$.

Verificações na decodificação

- ▶ tamanho = k e $c < n$;
- ▶ primeiro byte = 00;
- ▶ $IHash'$ igual ao SHA3-256 do rótulo;
- ▶ separador 01 depois dos zeros.

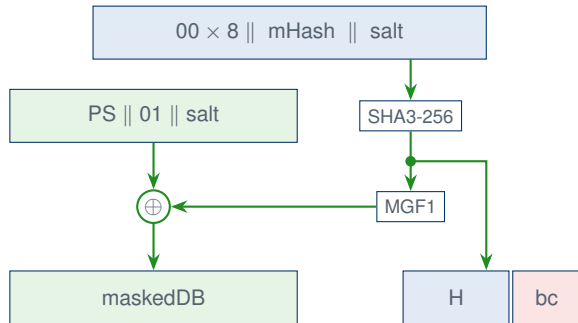
Uma única resposta

Toda falha gera falha na decifração. O laço percorre o bloco inteiro e o $IHash$ é comparado com `hmac.compare_digest`. Respostas diferentes seriam um oráculo (Manger, 2001).



cifrar $\approx 0,15$ ms decifrar $\approx 6,5$ ms

- ▶ 20 testes + 3 de interoperabilidade com a `pycryptodome`, nos dois sentidos.



$EM = \text{maskedDB} \parallel H \parallel bc$ (bit mais alto zerado) $\rightarrow s = EM^d \bmod n$

- ▶ mHash = SHA3-256 do **arquivo**.
- ▶ **salt** aleatório de 32 bytes: mesma entrada, assinaturas diferentes.
- ▶ Reusa a **MGF1** da Parte II.
- ▶ Assinatura de 256 bytes gravada em **Base64**.

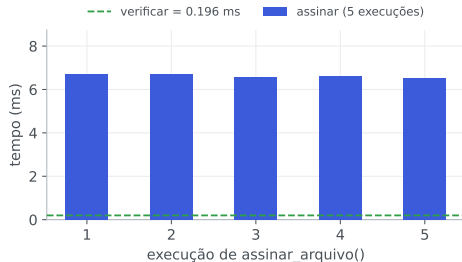
Abordagem rejeitada pelo roteiro

$$s = H(m)^d \bmod n$$

Determinística e herda a maleabilidade do RSA puro.

O que o PSS acrescenta

- ▶ salt aleatório e máscara MGF1 sobre todo o módulo;
- ▶ estrutura verificável: PS, 01, trailer 0xbc;
- ▶ prova de segurança ligada ao problema RSA.



assinar $\approx 6,6$ ms verificar $\approx 0,2$ ms

- ▶ 5 assinaturas do mesmo arquivo: **5 diferentes.**
- ▶ Teste garante que $s^e \bmod n \neq H(m)$.

Arquivo documento.txt.sig

```
-----BEGIN RSA-SEPCOMP SIGNATURE-----
{"formato": "rsa-segcomp-assinatura-v1",
 "algoritmo": "RSASSA-PSS",
 "hash": "SHA3-256",
 "mgf": "MGF1-SHA3-256",
 "tamanho_salt": 32,
 "arquivo": "documento.txt",
 "assinatura": "YB+vKXyb/uFd...=="}
-----END RSA-SEPCOMP SIGNATURE-----
```

1. Valida delimitadores, JSON e campos.
2. Algoritmo, hash e MGF comparados com valores fixos (sem *downgrade*).
3. Base64 decodificado com `validate=True`.
4. Recalcula o SHA3-256 do arquivo.
5. RSASSA-PSS-VERIFY com a chave pública.

Nenhuma exceção escapa: toda falha vira **ASSINATURA INVÁLIDA**.

Cenário	Resultado	Onde falha
Original (controle)	Válida	—
(a) 1 byte do arquivo	Inválida	hash H
(b) 1 byte da assinatura	Inválida	padding / H
(c) outra chave pública	Inválida	padding / H
(d) .sig malformatado	Inválida	parsing

Por que sempre falha

Um bit diferente no arquivo muda o SHA3-256 inteiro. Na assinatura ou na chave, $s^e \bmod n$ vira um bloco aleatório, sem $0xbc$, sem o 01 e sem o H correto.

RSA sem padding

- ▶ **Determinístico:** mesma mensagem, mesmo ciphertext (ataque de dicionário).
- ▶ **Maleável:** $E(m_1) E(m_2) = E(m_1 m_2)$; assinaturas também se combinam.
- ▶ **Mensagem pequena:** com $e = 3$ e $m^3 < n$, basta a raiz cúbica.

OAEP na cifragem

Seed aleatória e Feistel com MGF1: cifragem probabilística e adulteração detectada. Segurança IND-CCA2.

PSS na assinatura

Salt aleatório, MGF1 e estrutura verificável: não dá para forjar combinando assinaturas.

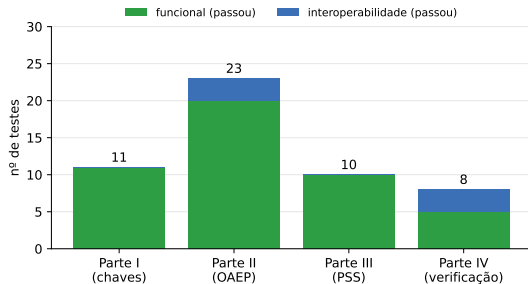
	RSA-PSS (2048 bits)	Ed25519
Problema difícil	fatoração / problema RSA	logaritmo discreto em curva elíptica
Segurança aproximada	~112 bits	~128 bits
Chave pública	256 bytes	32 bytes
Assinatura	256 bytes	64 bytes
Aleatoriedade	salt novo a cada assinatura	nonce determinístico
Assinar / verificar	lento / rápido	rápido / rápido

Vantagem do Ed25519

Chaves e assinaturas bem menores, e não depende de um bom gerador aleatório na hora de assinar.

Limite comum

Ambos são quebrados por computação quântica (Shor). Alternativas: ML-DSA e SLH-DSA.



Resultado da suíte

52/52

testes aprovados

- ▶ **PSS** ↔ cryptography, nos dois sentidos.
- ▶ **OAEP** ↔ pycryptodome: a cryptography não aceita SHA3 no OAEP.
- ▶ As bibliotecas só conferem a nossa saída.

Testes e demonstração

bash exemplos/demo.sh



terminal - bash exemplos/demo.sh

\$ bash exemplos/demo.sh

== 1. Geração de chaves RSA-2048 (Miller-Rabin) ==

Par de chaves RSA-2048 gravado em exemplos/saida/chaves/

== 2. RSA-OAEP ==

Ciphertext (Base64): cfYtY2GD0PV03CQRdxFtVkfXh4GqnWTWR3KI4eGu3cbgpBN7ilik0e/NIF2M...

senha: correct horse battery staple

Ciphertext adulterado:

Erro: falha na decifração

== 3. Assinatura RSA-PSS ==

Assinatura gravada em exemplos/saida/documento.txt.sig

-----BEGIN RSA-SEGCMP SIGNATURE-----

```
{ "formato": "rsa-segcomp-assinatura-v1", "algoritmo": "RSASSA-PSS", "hash": "SHA3-256", "mgf": "MGF1-SHA3-256",  
  "tamanho_salt": 32, "arquivo": "documento.txt", "assinatura": "RN/BvcLmdIYtMzk0b0eFzIvnNhFD5KZQczDm/vG+7ffUJB8i1  
iIuKhDQLF7s2LhNqFowtExcBjJkFfMCAAwR7yjtF+YfOXL+aRbukyBP0by3noF8J1GaRZGLn02vn5qFyvxye0l3H0RY+eGRf1nwbLXcL2Tr33zGW  
gLpxF7/bokT7RtUGIh0jdeivDzp9ubj2dW7L1PqC3hShcaFxDsIxxYp9D9EHbjWsajQtu1W15NZm6/h9CGR0yuiImzwkKNLrVRVd5niosWCltidX  
+XfEJYtB4UnKmoFOM6YTMvyOL9RyMDzm6S1K//udEj+1CKAif4mSLE0Ls0bzQu4Z0l8pw==" }
```

-----END RSA-SEGCMP SIGNATURE-----

== 4. Verificação ==

-- original:

ASSINATURA VÁLIDA: o arquivo é íntegro e foi assinado pela chave informada.

-- (a) um byte do arquivo alterado (1.000 -> 9.000):

ASSINATURA INVÁLIDA: o arquivo foi alterado, a assinatura foi adulterada ou a chave não corresponde.

-- (b) um byte da assinatura alterado:

ASSINATURA INVÁLIDA: o arquivo foi alterado, a assinatura foi adulterada ou a chave não corresponde.

-- (c) outra chave pública:

ASSINATURA INVÁLIDA: o arquivo foi alterado, a assinatura foi adulterada ou a chave não corresponde.

- ▶ Chaves RSA-2048 geradas em 0,365 s em média, com Miller-Rabin calibrado pela FIPS 186-5.
- ▶ O OAEP cifra até 190 bytes e rejeita qualquer adulteração com um único erro.
- ▶ O PSS produz assinaturas diferentes para o mesmo arquivo, verificáveis com a chave pública.
- ▶ Os três cenários de adulteração do roteiro foram rejeitados, e os 52 testes passaram.

Conclusão central

A segurança não vem do RSA isolado, e sim do **padding probabilístico**: OAEP para cifrar e PSS para assinar, cada um com seu papel.